SISTEMAS OPERATIVOS

Aula Teórica 9

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

Scheduling in Interactive Systems (4)

Priority Scheduling

Round-robin scheduling makes the implicit assumption that all processes are equally importante.

The need to take external factors into acount leads to **priority scheduling**.

The basic idea is straightforward: each process is assigned a priority, and the runnable process with the highest priority is allowed to run.

To prevent high-priority processes from running indefinitely, the scheduler may decrease the priority of the currently running process at each clock interrupt.

Alternatively, each process may be assigned a maximum time *quantum* that is allowed to run.

When this *quantum* is used up, the next-highest-priority process is given a chance to run.

After a process has been punished long enough, its priority needs to be raised by some algorithm, to let it run again.

Otherwise all processes will eventually end up at 0.

Priorities can be assign to processes statiscally or dinamically.

The UNIX system has a command, **nice**, which allows a user to voluntary reduce the priority of his process, in order to be nice to the other users.

Scheduling in Interactive Systems (5)

Priorities can also be assigned dynamically by the system to achieve certain system goals.

For example, some processes are highly I/O-bound and spend most of their time waiting for I/O to complete. Whenever such a process wants the CPU, it should be given the CPU immediately, to let it start its next I/O request. Making the I/O-bound process wait a long time for the CPU will just mean having it around occupying memory for an unnecessarely long time.

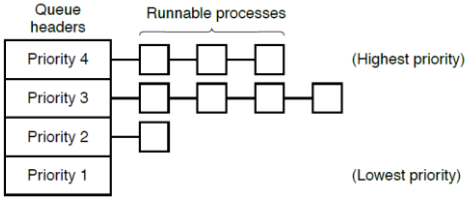


Figure 2-44. A scheduling algorithm with four priority classes.

A simple algorithm for giving good service to I/O-bound processes is to set the priority to $1/f$, where f is the fraction of the last *quantum* that a process used. A process that used only 1 msec of its 50 msec *quantum* would get priority 50, while a process that ran 25 msec before blocking would get priority 2, and a process that used the whole *quantum* would get priority 1.

Example: A system of four priority classes

The scheduling algorithm works as follows: as long as there are runnable processes in priority class 4, just run each one for one quantum, round-robin fashion, and never bother with lower-priority classes. If priority class is empty, then run the class 3 processes round-robin, if both class 4 and 3 are empty, then run class 2 round-robin, and so on.

If priorities are not adjusted occasionally, lower-priority classes may all starve to death.

Scheduling in Interactive Systems (6)

Multiple Queues
Scheduling in **UNIX** uses a multi-queue scheme, in which each queue corresponds to a range of priority values. Each process is associated with an integer priority value, and, unlike usual, a lower numeric priority value corresponds to a more prioritized process.

At the end of each quantum, the running process is placed at the end of the queue corresponding to its priority, and the dispatch routine is invoked, which puts the first process of the highest priority queue into execution.

In case there is more than one process in the highest priority queue, they are executed in rotation (round-robin); otherwise, the same process will continue running if there is no process in a higher priority queue.

The priority value in user mode varies dynamically taking into account the CPU usage history of the process, in order to benefit processes that have used less CPU recently.
Calculating the priority, the scheduler uses two fields of the *proc* structure:

p_cpu – An accumulator of the CPU time used by the process.
p_pri – The current priority value of the process.

Periodically (every 10 ms) the **system clock generates an interrupt**, a clock tick, whose handling routine increments the value of *p_cpu* of the running process by one unit.
Every 1 second, the priorities and CPU utilization times of all processes are recalculated, that is, **a rescheduling of the processes is performed by calculating their new priorities**.

Scheduling in Interactive Systems (7)

Scheduling in Linux

One of the limitations of the UNIX scheduling algorithm is its poor scalability with the number of processes running on the system. This is due to the fact that **every second** it is necessary to calculate the priorities of all processes, and this calculation takes time linear to the number of processes running on the machine.

The Linux scheduling algorithm, particularly in kernel versions 2.2 and 2.4, solves this problem through the introduction of the concept of an **epoch**. The system execution time is divided into consecutive epochs, and each epoch ends when all runnable processes have either finished using their respective quantum or opted not to use it.

In this algorithm, although priorities are changed whenever a process uses part of its *quantum*, this calculation is simple and incremental, that is, it consists only of updating the priority of each process as they use part of their quantum.
The quantum assigned to each process at the beginning of an epoch (measured in clock ticks) varies according to the time that this process did not use in the previous epoch:

$$Quantum_initial = Quantum_base + Quantum_previous_unused/2$$

Quantum_base is a configuration parameter that depends on the frequency of clock interrupts (a reference value is 20 ticks).
The calculation of priorities is also based on the amount of quantum remaining to be used, but in the current epoch.

$$Priority = Priority_base + Quantum_current_unused - nice$$

Scheduling in Interactive Systems (8)

The scalability of the Linux algorithm is improved compared to UNIX scheduling due to the fact that

- **the durations of each quantum are recalculated once per epoch and not periodically**,

and in turn,

- **the duration of the epoch increases as the number of simultaneously running processes increases**,

reducing the overhead of priority calculation on the total processing time.

However, the scalability of the scheduling algorithm of Linux 2.4 is still limited by the cost of the operation of recalculating the quantum value of all processes at the end of each epoch.

The computational cost of this operation is linear with the number of processes, that is, it has a complexity of O(n), which can become prohibitive in a system with a high number of runnable processes.

Additionally, the implementation of the operation of choosing the next process to execute by the scheduler also has O(n) complexity.

To solve this scalability problem, the scheduling algorithm in version 2.6 of the Linux kernel was developed from scratch, in a break with the previous version, and aims to run in constant time, with a complexity of **O(1)**.

Scheduling in Interactive Systems (9)

The algorithm was developed with a multi-queue scheduling with dynamic priorities.

It uses 140 priority levels in each of its two queues:

- the list of active processes;

- the list of expired processes,

and when invoked, the scheduler aims to choose the process from the highest priority level, alternating between them in a circular (round-robin) manner whenever there is more than one process at the same level.

From version 2.6.23 of the kernel, Linux started using a scheduling algorithm called Completely Fair Scheduler (CFS)

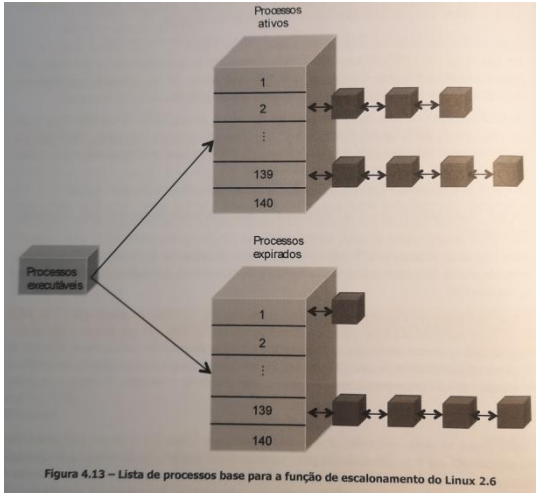


Figura 4.13 – Lista de processos base para a função de escalonamento do Linux 2.6

Figura retirada de "Sistemas Operativos", José Alves Marques, et al., FCA – Editora Informática

Scheduling in Interactive Systems (10)

AnotherAlgorithm
Implementing an algorithm that set up priority classes to processes, and
processes in the highest class were run for one quantum,
Processes in the next-highest class were run for two quanta,
Processes in the next class, were run for four quanta,
Processes in the next classe, were run for eight quanta,
And so on, 16, 32, 64, ….
When a process used up all the quanta allocated to it, it was removed down one class.
Thus , the processes in the highest class would run more frequently and with high priority, BUT FOR A SHORT TIME – ideal for interactive processes.

Example: Consider a process that needed to compute continuously for 100 quanta.
It would initially be given one quantum, them swapped out.
Next time it would get two quanta before being swapped out.,
On succeeding runs it would get 4, 8, 16, 32, and 64 quanta, although it would have used only 37 of the final 64 quanta to complete its work.
Only 7 swaps would be needed instead of the 100 CPU-bound with a pure round-robin algorithm.

Furthermore, as the process sank deeper and deeper into the priority queues, it would be run less and less frequently, saving the CPU for short, interactive processes.

Scheduling in Interactive Systems (11)

Shortest Process Next

Because shortest job first always produces the minimum average response time for batch systems, it would be nice if it could be use for interactive processes as well.

Interactive processes generally follow the pattern of wait for command, execute command, wait for command, execute command, etc.

If we regards the execution of each command as a separete "job", then we can minimize overall response time by running the shortest one first. One approach is to make estimates based on past behavior and run the processes with the shortest estimated running time.

Guaranteed Scheduling

A completely different approach to scheduling is to make real promises to the users about performance an then live up to those promises. One promise that is realistic to make and easy to live up to is this:

If *n* users are logged in while you are working, you will receive about 1/*n* of the CPU power.

Similarly, on a single-user system with *n* processes running, all things being equal, each one should get 1/*n* of the CPU cycles.

To make this promise valid, the system must keep track of how much CPU each process has had since its creation. Doing this, it is fairly straightforward to compute the ratio of actual CPU time consumed to CPU time entitled.

A variant of such a scheduling regime is used in Linux CFS (Completely Fair Scheduler) algorithm which keeps track of the "spent execution time".

Lottery Schedulling

While making promises to the users and then living up to them is a fine idea, it is difficult to implement. However, another algorithm can be used to give similarly predictable results with much simpler implementation.

The basic idea is to give processes lottery implementation tickets for various system resource, such as CPU time. Whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the resource.

To paraphrase George Orwell: "All processes are equal, but some are more equal".

More important processes can be given extra tickets, to increase their odds of winning.

Lottery scheduling has several interesting properties.
For example, is a new process shows up and is granted some tickets, at the very next lottery it will have a chance of winning in proportion to the number of tickets it holds. Lottery scheduling is highly responsive.

Cooperating processes may exchange tickets if they wish.
For example, when a client process sends a message to a server process and then blocks, it may give all of its tickets to the server, to increase the chance of the server running next. When the server is finished, it returns the tickets so that the client can run again.

Lottery scheduling can be used to solve problems that are difficult to handle with other methods.

Scheduling in Interactive Systems (13)

Fair-Share Scheduling

So far we have assumed that each process is scheduled on its own, without regard to who its owner is.
As a result, if user 1 starts up nine processes and user 2 starts up one process, with round-robin or equal priorities, user 1 will get 90% of the CPU and user 2 only 10% of it.

To prevent this situation, some systems take into account which user owns a process before scheduling.
In this model, each user is allocated some fraction of the CPU and the schedular picks processes in such a way as to enforce it.

Example: Consider a system with two users, each of which has been promised 50% of the CPU. User 1 has four processes, A, B, C, D, and user 2 has only one, E. If round-robin scheduling is used, the possible scheduling sequence that meets all constraints is the one:

A E B E C E D E A E B E C E D E …

On the other hand, if user 1 is entitled to twice as much CPU time as user 2, we might have:

A B E C D E A B E C D E …

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

